Kaylee Alvey

Lab Assignment 1 Report

a) The assignment for this lab was to write a program that would calculate the hamming distance between the binary representation of two ascii strings. I knew as far as logic went that in order to calculate the hamming distance between two binary numbers, you have two xor the two binary numbers, and the number of 1s in the result would be the hamming distance. I also knew that we would have to loop through the individual bits of the xor result in order to count the number of 1s. When I started coding, I realized that the number of bits we were able to process at once was limited by the size of the registers. In order to get around this, I incorporated code into my loop that would xor each string one byte at a time, and for each xor it would loop through the xor result, and increment a counter by 1 every time it saw a 1. This loop would continue until one of the strings ran out of bytes, which means that the register I was pushing the bytes into would store 0, the null terminator.

b)

```
section .data
    newline db 0xA  ; Newline character for output formatting

    msg1 db 'The hamming distance is ', 0x0A    ;setting up to display
distance
    len1 equ $-msg1

    string1 db 'foo', 0
    string2 db 'bar', 0

    ;string1 db 'this is a test', 0
    ;string2 db 'of the emergency broadcast', 0

    ;string1 db 'computer', 0
    ;string2 db 'engineer', 0

section .bss
    distance resb 10  ; Reserve space for up to 10 bytes (Hamming
distance)

section .text
    global _start

_start:
    mov ecx, 0 ; i of loop
```

```
    mov edx, 0 ;counter that goes up

.loop:

    ; Load the strings byte by byte
    mov al, byte[string1+ecx]  ; Binary number 1
    mov bl, byte[string2+ecx]  ; Binary number 2

    cmp al, 0
    je .exit
    cmp bl, 0
    je .exit

    inc ecx

    ; XOR the two numbers to compare bits (find differing bits)
    xor al, bl     ; EAX = EAX XOR EBX
    jmp .count_ones


.count_ones:
    test al, 1      ; Test the least significant bit of EAX
    jz .no_bit       ; If it's 0, skip to the next bit
    inc edx          ; Increment the counter (bit is 1)

.no_bit:
    shr al, 1       ; Shift EAX right to check the next bit
    jnz .count_ones  ; Repeat until all bits are checked
    jmp .loop

.exit:
    ; Move Hamming distance into EAX to convert to string
    mov eax, edx         ; Move the distance of the Hamming distance into
eax
    mov ebx, distance    ; Point EBX to the distance buffer
    add ebx, 10          ; Move to the end of the buffer (to fill from
right)
    mov byte [ebx], 0    ; Null-terminate the buffer
    dec ebx              ; Point to the last character
```

```
.convert_to_ascii:
    xor edx, edx            ; Clear remainder
    mov ecx, 10             ; Set divisor to 10
    div ecx                 ; EAX = EAX / 10, remainder in EDX
    add dl, '0'             ; Convert remainder to ASCII ('0' = 48)
    mov [ebx], dl           ; Store the digit in the buffer
    dec ebx                 ; Move to next character position
    test eax, eax           ; Check if quotient is zero
    jnz .convert_to_ascii   ; If not zero, continue conversion

    ; Print msg1
    mov eax, 4
    mov ebx, 1
    lea ecx, msg1
    mov edx, len1
    int 0x80

    ; Print the distance (Hamming distance) as string
    mov eax, 4
    mov ebx, 1
    lea ecx, [distance]
    mov edx, 10
    int 0x80

    ; Print a newline character
    mov eax, 4
    mov ebx, 1
    lea ecx, [newline]
    mov edx, 1
    int 0x80

    ; Exit the program
    mov eax, 1
    xor ebx, ebx
    int 0x80
```

c)
Compiled and ran 3 times, once with each set of string1 and string2. First ‘foo’ and ‘bar, then ‘this is a test’ and ‘of the emergency broadcast’, and last ‘computer’ and ‘engineer’